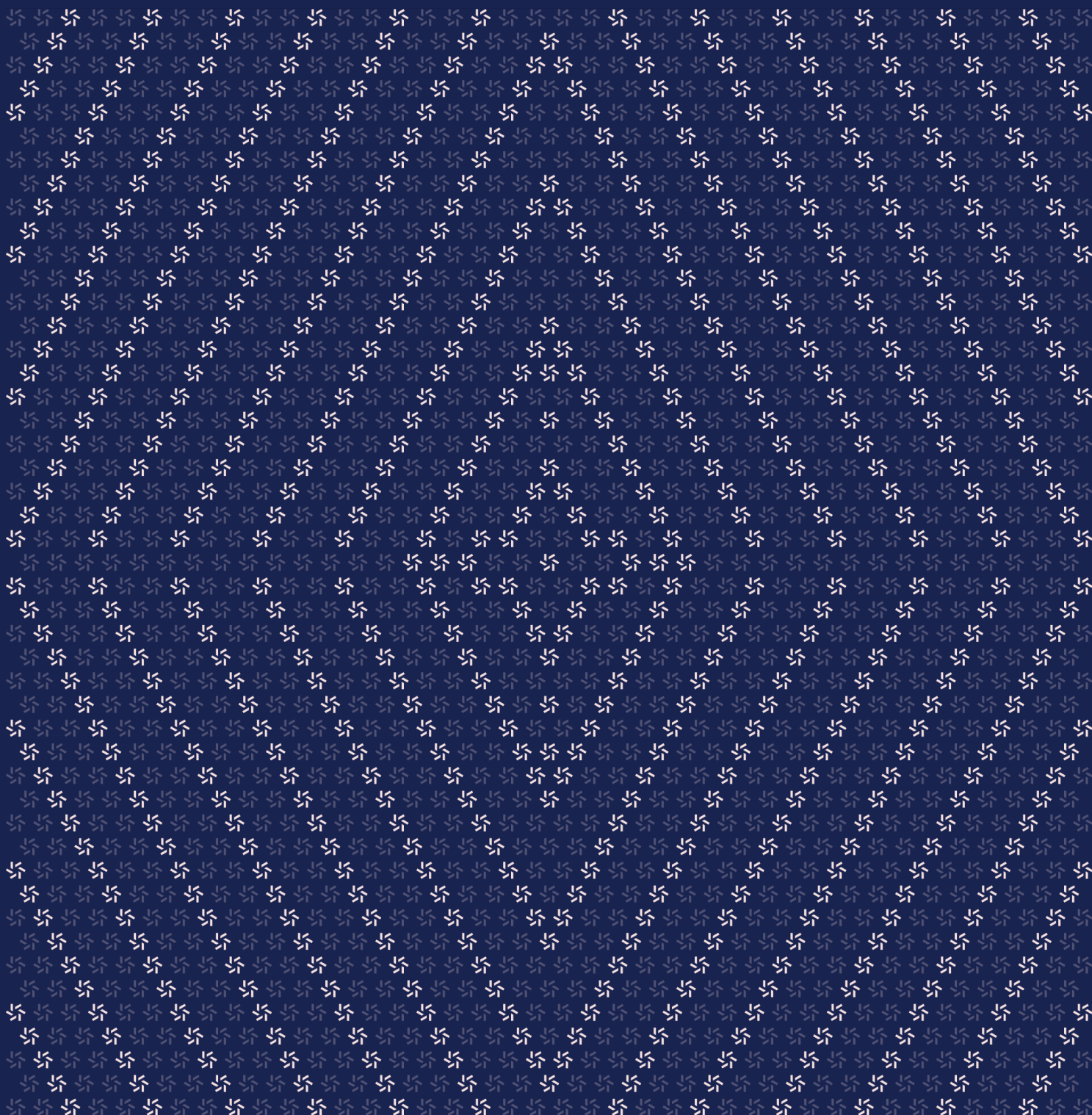


July 31, 2026

FSP diff - FCC

Smart Contract Security Assessment



Contents

About Zellic	4
1. Overview	5
1.1. Executive Summary	5
1.2. Goals of the Assessment	5
1.3. Non-goals and Limitations	6
1.4. Results	6
2. Introduction	8
2.1. About FSP diff - FCC	8
2.2. Methodology	9
2.3. Scope	10
2.4. Project Overview	14
2.5. Project Timeline	14
3. Detailed Findings	15
3.1. Machines running a disabled TEE version remain eligible for routing and signature verification	15
3.2. Reissue flow can get stuck when batch payments are identical	18
3.3. Safe-backed governance snapshots can be satisfied without the snapshot signers' consent	20
3.4. Stale healthy availability proofs can suspend production TEE machines when challengeValidityDurationSeconds is configured too high	22
3.5. Freshly registered TEEs cannot restore older backups	24

3.6.	Failed availability proofs are not invalidated by a later successful availability confirmation	26
4.	System Design	27
4.1.	FlareTeeManager: TEE machine lifecycle	27
4.2.	FlareTeeManager: wallet key management	28
4.3.	MachinePathManagerFacet path list flow	30
4.4.	FlareTeeManager: instruction dispatch	30
4.5.	TeePayments: payment contracts	31
4.6.	TeePaymentsUtxo batch settings	32
4.7.	TeePaymentsFeeScheduleManager schedules	33
4.8.	TeePaymentsConfigVerifier proof verification	33
5.	Assessment Results	34
5.1.	Disclaimer	34

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the [#1 CTF \(competitive hacking\) team](#) [↗] worldwide in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io [↗] and follow [@zellic_io](#) [↗] on Twitter. If you are interested in partnering with Zellic, contact us at hello@zellic.io [↗].



1. Overview

1.1. Executive Summary

Zellic conducted a security assessment for Flare Foundation from June 22nd to July 22nd, 2026. During this engagement, Zellic reviewed FSP diff - FCC's code for security vulnerabilities, design issues, and general weaknesses in security posture.

1.2. Goals of the Assessment

In a security assessment, goals are framed in terms of questions that we wish to answer. These questions are agreed upon through close communication between Zellic and the client. In this assessment, we sought to answer the following questions:

- UTXO/BTC payments: is the per-anchor nonce stream and cyclic round-robin anchor selection correct under concurrent batches, anchor growth (grow-only, prefix-preserving superset check), and the reuse-window accounting (availableAt reserved at open, only ever brought earlier on close)? Can an anchor be double-spent or a batch opened on a busy anchor?
- UTXO/BTC payments: is batch open/close (ended vs reward-epoch-changed vs full) and the reissue/replacement state machine (start vs continue an attempt, _checkPaymentRange staying within the batch window, finalize when emittedCount == paymentCount) free of stuck, skipped, or replayed payments?
- Account payments/XRP: is the native-nonce derivation (initialNonce + paymentId - 1) and the reissue path (validating the supplied instruction against the stored paymentHash before any external call) correct and non-replayable?
- Both payment models: is pay/reissue authorization (authorizationAddress per account vs wallet-owner for registration/addAnchors) and the fee pre-flight (FeeTooLow, getPaymentFee) correct, and is the config-verifier validate-only contract (reads verified fields straight from calldata proof) safe against a malformed or replayed FDC2 proof?
- Verification: do Fdc2ProofVerification and the SignedPayload envelope bind chainid, extension/source, and the right domain prefix so TEE-signed responses, payment-config attestations, and machine-path-list signatures cannot be replayed across chains, extensions, or message types?
- TEE machines: can the lifecycle be abused - registering with a forged or stale attestation, escaping a pause, or registering for an extension without being allowlisted? Is the permissionless pause of machines running disabled code hashes / expired availability correctly gated by the post-unpause grace window?
- Diamond/governance: are diamondCut, the ERC-7201 storage namespacing, and FlareTeeManagerInit init-once correct and collision-free? Is every governance- only setter (diamond governance, per-extension setNewTeeGovernance, owner allowlists, setExtensionOperator including the id-0 special case) correctly access-controlled, and can the extension operator only drive inert prep steps?
- FCE/applications: is application registration and isolation sound - can an FCE application (its owner, owner allowlist, governance signer set, supported versions/key types, and

instructions sender) only touch its own TEE machines and wallets, and never reach into another application or into PMW/FDC2? Are the extension-id ranges and the ownership-transfer allowlist gating enforced correctly?

- Wallets/keys: do the Shamir backup/restore flows, key admin thresholds, and direct backup/restore (gated by the active machine-path list, latest-nonce-wins deprecation) preserve key confidentiality and prevent unauthorized restore?
 - FDC2: is `Fdc2Hub.requestAttestation` threshold/TEE-set validation correct, and does the fee/inflation/reward-offer accounting (`Fdc2RewardOffersManager`, `Fdc2RequestFeeConfigurations`, `Fdc2InflationConfigurations`) match the existing FDC/FTSO reward model it mirrors?
 - Integration with FSP: are the reward-offer flows into `RewardManager` and the reward-epoch / signing-policy reads from `FlareSystemsManager` and `Relay` used consistently with the already-audited FSP contracts?
-

1.3. Non-goals and Limitations

We did not assess the following areas that were outside the scope of this engagement:

- Front-end components
- Infrastructure relating to the project
- Key custody

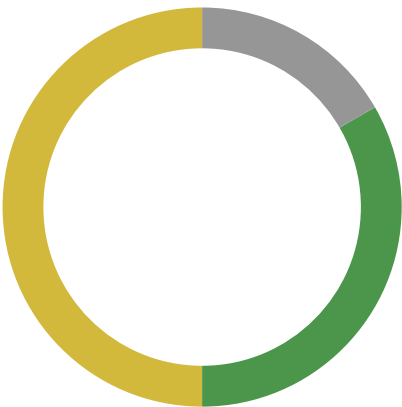
Due to the time-boxed nature of security assessments in general, there are limitations in the coverage an assessment can provide.

1.4. Results

During our assessment on the scoped FSP diff - FCC contracts, we discovered six findings. No critical issues were found. Three findings were of medium impact, two were of low impact, and the remaining finding was informational in nature.

Breakdown of Finding Impacts

Impact Level	Count
Critical	0
High	0
Medium	3
Low	2
Informational	1



2. Introduction

2.1. About FSP diff - FCC

Flare Foundation contributed the following description of FSP diff - FCC:

Flare Confidential Compute (FCC) extends Flare with Trusted Execution Environments (TEEs): attested, isolated compute units that run specified code, custody cryptographic keys, and produce signed responses to instructions submitted on Flare. FCC is a new protocol within the Flare Systems Protocol (FSP), alongside FDC and FTSO, added on top of the previously Zellic-audited FSP codebase. Like those protocols it relies on FSP entity participation - data providers (voters) run a relay client that signs each instruction with their FSP signing-policy key.

FCC is an umbrella. The smart contracts let allowlisted operators register and manage TEE machines (register them with an attestation, move them through a lifecycle of initialized -> production -> paused), custody protocol wallets and keys across those machines, and dispatch signed instructions to them. The registered TEE machines then run different applications under the FCC umbrella - currently FCE, PMW, and FDC2, with more possible over time:

- FCE (Flare Confidential Extensions) - lets a team register its own application on FCC-registered TEE machines and run custom confidential compute, with its own owner, governance signer set, owner allowlist, supported TEE code-hash/platform versions and key types, and instructions-sender contract. Implemented by ExtensionManagerFacet, ExtensionGovernanceFacet, and OwnerAllowlistFacet (all in scope).
- PMW (Protocol-Managed Wallet) - multisig wallets on external chains whose key shares are custodied across TEE machines, with on-chain bookkeeping for wallets, key admins, and payment streams. PMW supports an account model (TeePayments, e.g. XRPL) and a UTXO model (TeePaymentsUtxo, e.g. BTC) with per-anchor nonce streams, batched payments, and batch reissue/replacement. The BTC/UTXO payment contracts are in scope.
- FDC2 (Flare Data Connector v2) - the TEE-based update to FDC that FCC brings. It serves the same attestation-type catalog as legacy FDC but routes each request to a set of TEE machines that each verify and sign the response, instead of a per-round bit-vote and Merkle finalization. Fdc2Hub, Fdc2Verification, and the shared Fdc2ProofVerification library (also used by PMW payment-config attestations) are in scope.

The on-chain side lives in contracts/tee and is a single EIP-2535 diamond proxy (FlareTeeManager) with around 22 facets and 20 libraries over ERC-7201 namespaced storage. Around the diamond sit several UUPS-upgradeable contracts (the TeePayments suite, TeeRewardOffersManager, VrfVerifier).

The protocol specification is at <https://gitlab.com/flarenetwork/docs-team/flare-specs/-/tree/fcc> and an in-repo spec set is under docs/specs/FCC.

2.2. Methodology

During a security assessment, Zellic works through standard phases of security auditing, including both automated testing and manual review. These processes can vary significantly per engagement, but the majority of the time is spent on a thorough manual review of the entire scope.

Alongside a variety of tools and analyzers used on an as-needed basis, Zellic focuses primarily on the following classes of security and reliability issues:

Basic coding mistakes. Many critical vulnerabilities in the past have been caused by simple, surface-level mistakes that could have easily been caught ahead of time by code review. Depending on the engagement, we may also employ sophisticated analyzers such as model checkers, theorem provers, fuzzers, and so on as necessary. We also perform a cursory review of the code to familiarize ourselves with the contracts.

Business logic errors. Business logic is the heart of any smart contract application. We examine the specifications and designs for inconsistencies, flaws, and weaknesses that create opportunities for abuse. For example, these include problems like unrealistic tokenomics or dangerous arbitrage opportunities. To the best of our abilities, time permitting, we also review the contract logic to ensure that the code implements the expected functionality as specified in the platform's design documents.

Integration risks. Several well-known exploits have not been the result of any bug within the contract itself; rather, they are an unintended consequence of the contract's interaction with the broader DeFi ecosystem. Time permitting, we review external interactions and summarize the associated risks: for example, flash loan attacks, oracle price manipulation, MEV/sandwich attacks, and so on.

Code maturity. We look for potential improvements in the codebase in general. We look for violations of industry best practices and guidelines and code quality standards. We also provide suggestions for possible optimizations, such as gas optimization, upgradability weaknesses, centralization risks, and so on.

For each finding, Zellic assigns it an impact rating based on its severity and likelihood. There is no hard-and-fast formula for calculating a finding's impact. Instead, we assign it on a case-by-case basis based on our judgment and experience. Both the severity and likelihood of an issue affect its impact. For instance, a highly severe issue's impact may be attenuated by a low likelihood. We assign the following impact ratings (ordered by importance): Critical, High, Medium, Low, and Informational.

Zellic organizes its reports such that the most important findings come first in the document, rather than being strictly ordered on impact alone. Thus, we may sometimes emphasize an "Informational" finding higher than a "Low" finding. The key distinction is that although certain findings may have the same impact rating, their *importance* may differ. This varies based on various soft factors, like our clients' threat models, their business needs, and so on. We aim to provide useful and actionable advice to our partners considering their long-term goals, rather than a simple list of security issues at present.

2.3. Scope

The engagement involved a review of the following targets:

FSP diff - FCC Contracts

Type	Solidity
Platform	EVM-compatible

Target Primary Review Period: Only changes between 264dab7...6fb098a

Repository <https://github.com/flare-foundation/flare-smart-contracts-v2> ↗

Version 6fb098a6cf17168426b1d91686e0810d10a9d8bd

Programs

- diamond/facets/DiamondLoupeFacet.sol
- diamond/implementation/Diamond.sol
- diamond/libraries/LibDiamond.sol
- fdc2/implementation/Fdc2Hub.sol
- fdc2/implementation/Fdc2InflationConfigurations.sol
- fdc2/implementation/Fdc2RequestFeeConfigurations.sol
- fdc2/implementation/Fdc2RewardOffersManager.sol
- fdc2/implementation/Fdc2Verification.sol
- fdc2/library/Fdc2ProofVerification.sol
- fdc2/proxy/Fdc2HubProxy.sol
- fdc2/proxy/Fdc2InflationConfigurationsProxy.sol
- fdc2/proxy/Fdc2RequestFeeConfigurationsProxy.sol
- fdc2/proxy/Fdc2RewardOffersManagerProxy.sol
- fdc2/proxy/Fdc2VerificationProxy.sol
- fdc2/structs/Fdc2Structs.sol
- tee/diamond/FlareTeeManager.sol
- tee/facets/DiamondGovernanceFacet.sol
- tee/facets/ExtensionGovernanceFacet.sol
- tee/facets/ExtensionManagerFacet.sol
- tee/facets/ExternalAddressesFacet.sol
- tee/facets/FlareTeeManagerInit.sol
- tee/facets/InstructionsFacet.sol
- tee/facets/MachineEmergencyPauseFacet.sol
- tee/facets/MachineManagerFacet.sol
- tee/facets/MachinePathManagerFacet.sol
- tee/facets/OperationFeesFacet.sol
- tee/facets/OwnerAllowlistFacet.sol
- tee/facets/VerificationFacet.sol
- tee/facets/VrfFacet.sol
- tee/facets/WalletBackupManagerFacet.sol
- tee/facets/WalletKeyManagerFacet.sol
- tee/facets/WalletManagerFacet.sol
- tee/facets/WalletProjectManagerFacet.sol
- tee/facets/WalletProjectPauseFacet.sol
- tee/implementation/TeePayments.sol
- tee/implementation/TeePaymentsBase.sol
- tee/implementation/TeePaymentsConfigVerifier.sol
- tee/implementation/TeePaymentsFeeScheduleManager.sol
- tee/implementation/TeePaymentsRegistry.sol
- tee/implementation/TeePaymentsUtxo.sol
- tee/implementation/TeeRewardOffersManager.sol
- tee/implementation/VrfVerifier.sol

Target	Primary Review Period Continued: Only changes between 264dab7...6fb098a
Repository	https://github.com/flare-foundation/flare-smart-contracts-v2 ↗
Version	6fb098a6cf17168426b1d91686e0810d10a9d8bd
Programs	tee/library/ExtensionGovernance.sol tee/library/ExtensionManager.sol tee/library/ExternalAddresses.sol tee/library/Instructions.sol tee/library/MachineEmergencyPause.sol tee/library/MachineManager.sol tee/library/MachinePathManager.sol tee/library/OperationFees.sol tee/library/OwnerAllowlist.sol tee/library/SystemStateVerifier.sol tee/library/Verification.sol tee/library/Vrf.sol tee/library/WalletKeyManager.sol tee/library/WalletManager.sol tee/library/WalletProjectManager.sol tee/library/WalletProjectPause.sol tee/library/XrplAddress.sol tee/proxy/TeePaymentsConfigVerifierProxy.sol tee/proxy/TeePaymentsFeeScheduleManagerProxy.sol tee/proxy/TeePaymentsProxy.sol tee/proxy/TeePaymentsRegistryProxy.sol tee/proxy/TeeRewardOffersManagerProxy.sol tee/structs/TeeInstructionsStructs.sol tee/structs/TeeMachinePathStructs.sol tee/structs/TeeMachineStructs.sol tee/structs/TeePaymentsStructs.sol tee/structs/TeeStructs.sol tee/structs/TeeVerificationStructs.sol tee/structs/TeeVrfStructs.sol tee/structs/TeeWalletStructs.sol governance/implementation/FlareGovernedAccess.sol governance/implementation/FlareGovernedBase.sol governance/implementation/FlareUpgradeableBase.sol governance/lib/FlareGovernance.sol utils/lib/SignedPayload.sol customFeeds/implementation/SFlrCustomFeed.sol fastUpdates/implementation/*.sol fastUpdates/lib/*.sol fdc/implementation/*.sol fscV1/implementation/*.sol ftso/implementation/*.sol inflation/implementation/*.sol

Target Primary Review Period Continued: Only changes between 264dab7...6fb098a

Repository <https://github.com/flare-foundation/flare-smart-contracts-v2> ↗

Version 6fb098a6cf17168426b1d91686e0810d10a9d8bd

Programs

- governance/implementation/Governed.sol
- governance/implementation/GovernedBase.sol
- governance/implementation/GovernedProxyImplementation.sol
- protocol/implementation/EntityManager.sol
- protocol/implementation/FlareSystemsCalculator.sol
- protocol/implementation/FlareSystemsManager.sol
- protocol/implementation/FtsoV2.sol
- protocol/implementation/FtsoV2Proxy.sol
- protocol/implementation/RewardManager.sol
- protocol/implementation/RewardOffersManagerBase.sol
- protocol/implementation/Submission.sol
- protocol/implementation/VoterPreRegistry.sol
- protocol/implementation/VoterRegistry.sol
- protocol/implementation/WNatDelegationFee.sol
- protocol/implementation/Relay.sol
- protocol/implementation/NodePossessionVerifier.sol
- protocol/lib/*.sol
- utils/implementation/AddressUpdatable.sol
- utils/implementation/TokenPoolBase.sol
- utils/lib/AddressSet.sol

Target Secondary Review Period: Only changes between 0212388...f4aad8d

Repository <https://github.com/flare-foundation/flare-smart-contracts-v2> ↗

Version f4aad8d5ff3c3e63d067ed95961935174f7dba6b

Programs

- facets/ExtensionGovernanceFacet.sol
- facets/MachinePathManagerFacet.sol
- library/ExtensionGovernance.sol
- library/MachinePathManager.sol

2.4. Project Overview

Zellic was contracted to perform a security assessment for a total of seven person-weeks. The assessment was conducted by two consultants over the course of five calendar weeks.

Contact Information

The following project manager was associated with the engagement:

Jacob Goreski
 ↗ Engagement Manager
jacob@zellic.io ↗

The following consultants were engaged to conduct the assessment:

Weipeng Lai
 ↗ Engineer
weipeng.lai@zellic.io ↗

Ayaz Mammadov
 ↗ Engineer
ayaz@zellic.io ↗

2.5. Project Timeline

The key dates of the engagement are detailed below.

June 22, 2026 Start of primary review period

June 26, 2026 Kick-off call

July 22, 2026 End of primary review period

August 4, 2026 Start of secondary review period

August 5, 2026 End of secondary review period

3. Detailed Findings

3.1. Machines running a disabled TEE version remain eligible for routing and signature verification

Target	ExtensionManagerFacet		
Category	Business Logic	Severity	Medium
Likelihood	Medium	Impact	Medium

Description

When an extension owner disables a TEE software version through `disableCodeHashPlatforms`, the function only sets the corresponding `codeHashPlatformDisabled` flags. It does not change the status of machines already running that version or remove them from the active-machine sets.

```
for (uint256 i = 0; i < _platforms.length; i++) {
    bytes32 platform = _platforms[i];
    // [...]
    extension.codeHashPlatformDisabled[_codeHash][platform] = true;
}
```

Consequently, a machine that was in `PRODUCTION` when its version was disabled remains in `PRODUCTION` and stays in both `extensionActiveTeeIds` and `activeTeeIds`. The pause function becomes permissionless for such a machine, but pausing still requires a separate transaction for each affected machine:

```
if (msg.sender == state.owner ||
    ExtensionManager.isCodeHashPlatformDisabled(state.extensionId,
        state.codeHash, state.platform))
{
    MachineManager.checkTeeStatus(state.status, TeeStatus.PRODUCTION,
        TeeStatus.SUSPENDED);
    newStatus = TeeStatus.PAUSED;
}
```

Until those transactions execute, random routing continues to treat the machines as active. The `getRandomTeeIds` function samples directly from `extensionActiveTeeIds` without checking whether each machine's (`codeHash`, `platform`) pair remains supported:

```
EnumerableSet.AddressSet storage set = s.extensionActiveTeeIds[_extensionId];
// [...] reservoir sampling
for (uint256 i = 0; i < _count; i++) {
```

```
        _teeIds[i] = set.at(indices[i]);  
    }
```

The Fdc2Hub contract uses this function when a requester does not supply explicit TEE IDs, so a machine running a disabled version can be selected by requestAttestation. The instruction path does not correct the selection. For regular operation types, Instructions.sendInstructions checks only that each destination is in PRODUCTION; for system operation types, including FDC2 attestation instructions, it skips even that status check. Neither branch verifies that the destination's software version remains supported.

```
bool _isSystemOpType = isSystemOpType(_instructionParams.opType);  
for (uint256 i = 0; i < _teeIds.length; i++) {  
    address teeId = _teeIds[i];  
    require(  
        i == 0 || MachineManager.getExtensionId(teeId) == extensionId,  
        ITeeCommonErrors.ExtensionIdMismatch()  
    );  
    if (!_isSystemOpType) {  
        require(  
            MachineManager.getTeeMachineStatus(teeId) ==  
                IMachineManager.TeeStatus.PRODUCTION,  
            ITeeCommonErrors.TeeMachineNotAvailable()  
        );  
    }  
    teeMachines[i] = MachineManager.getTeeMachine(teeId);  
}
```

Fdc2Verification also relies on the stale machine status. The _verifyTeeSignature function requires the signer to belong to the system extension and to remain in PRODUCTION, but it does not check the signer's code hash or platform:

```
require(  
    _flareTeeManager.getExtensionId(_signingTeeId) == 0,  
    InvalidTeeMachineExtensionId()  
);  
require(  
    _flareTeeManager.getTeeMachineStatus(_signingTeeId) ==  
        IMachineManager.TeeStatus.PRODUCTION,  
    TeeMachineNotAvailable()  
);
```

Although a disabled machine cannot renew its availability through confirmAvailability, expiry does not automatically move it out of PRODUCTION. Signature verification also does not check availability validity. The machine therefore remains routable, and its signatures remain valid, until a caller explicitly pauses it. Protocol correctness consequently depends on an off-chain bot

observing `CodeHashPlatformsDisabled` and pausing every affected machine before another request or proof is processed.

Impact

A retired TEE implementation can continue receiving paid instructions, being selected for FDC2 requests, and producing signatures accepted by `Fdc2Verification`. The exposure lasts until every affected machine is paused and can persist indefinitely if the off-chain bot fails.

Recommendations

We recommend making version retirement effective atomically. When `disableCodeHashPlatforms` disables a `(codeHash, platform)` pair, it should also move every active machine using that pair to `PAUSED` before the transaction completes. This preserves the invariant that every member of `extensionActiveTeeIds` and `activeTeeIds` runs a supported version.

The simplest implementation iterates over the extension's active machines and pauses each matching machine. This requires no new storage, but its gas cost scales with the extension's entire active machines.

The more efficient variant maintains a dedicated index of active machines keyed by their `(codeHash, platform)` pair, updated alongside the existing active sets in `changeStatus`. A natural place is a `mapping(bytes32 codeHash => mapping(bytes32 platform => EnumerableSet.AddressSet))` in the `MachineManager` state, into which a machine is inserted when it enters `PRODUCTION` and from which it is removed when it leaves. With this index in place, `disableCodeHashPlatforms` iterates only over the machines that actually run the disabled pair and moves each of them to `PAUSED`, so the work scales with the number of affected machines rather than the extension's entire active machines.

Because each extension is expected to have only a small number of active machines, on the order of 10 to 20, either approach should fit within normal block gas limits. To guard against a pathological case where the iteration grows too large to execute in a single transaction, the extension owner can ban some machines first and then disable the remaining, keeping the iteration small enough to complete.

Remediation

This issue has been acknowledged by Flare Foundation, and a fix was implemented in commit [87036ab4](#).

3.2. Reissue flow can get stuck when batch payments are identical

Target	TeePaymentsUtxo		
Category	Business Logic	Severity	Medium
Likelihood	Medium	Impact	Medium

Description

The reissue function decides if a call starts a new replacement attempt or continues an ongoing one by comparing the first supplied payment instruction against the batch's first payment.

```
_replacement = activeReplacements[_accountHash][_batchPaymentId];
if (_paymentInstructionMatches(_accountHash, _batchPaymentId,
    _firstPaymentInstruction)) {
    uint64 replacementId
    = ++replacementCounters[_accountHash][_batchPaymentId];
    _replacement.id = replacementId;
    _replacement.nextPaymentId = _batchPaymentId;
    _replacement.emittedCount = 0;
    // [...]
    return _replacement;
}
require(_replacement.id != 0 && !_replacement.finalized,
    NoActiveReplacement());
```

Payment instructions are not required to be unique. If all payments in a batch have the same fields (recipient, token, amount, max fee, and payment reference), then any continuation call also matches the batch's first payment. The contract then treats the continuation as a new attempt and restarts from the beginning.

If such a batch is too large to reissue in a single call, the replacement attempt can never reach the end of the batch. Every call restarts the attempt, so the reissue flow is stuck.

Impact

A batch of identical payment instructions that needs more than one reissue call can never finalize its replacement attempt. Flare noted that the paymentReference field is meant to be unique in their own usage and in the planned FAssets v2 integration, but this is not enforced on chain, so a protocol that does not use unique references can hit this state.

Recommendations

Consider one of the following:

- Add an explicit boolean parameter to `reIssue` that states if the call starts a new replacement attempt or continues the ongoing one.
- Change the order of the checks: if an active attempt is open and the first supplied payment matches the payment at `nextPaymentId`, continue the attempt, otherwise start a new one.

Remediation

This issue has been acknowledged by Flare Foundation, and a fix was implemented in commit [08940d3b](#).

3.3. Safe-backed governance snapshots can be satisfied without the snapshot signers' consent

Target	MachinePathManagerFacet		
Category	Business Logic	Severity	Medium
Likelihood	Low	Impact	Medium

Description

The `setNewTeeGovernanceSafe` function in `ExtensionGovernanceFacet` reads a Safe's owner set and threshold live and freezes them under a content-derived governance hash. A TEE machine commits to that hash at registration, so the frozen owner set and threshold are the anchors a bound machine trusts, and `Governance.md` notes that snapshot keys stay trusted for old-bound machines.

A machine path list becomes signed, and therefore active, only once every involved governance has approved it. For a Safe-backed governance that approval is the Safe calling `approveMachinePathList`, which marks a snapshot `safeApproved` on the strength of `isSnapshotSatisfiable` alone.

```
function isSnapshotSatisfiable(
    uint256 _extensionId,
    bytes32 _governanceHash,
    ISafeMinimal _safe,
    uint256 _liveThreshold
)
    internal view
    returns (bool)
{
    TeeGovernanceData storage teeGov
    = getState().extensionStates[_extensionId]
      .governanceHashToTeeGovernance[_governanceHash];
    uint256 snapshotThreshold = teeGov.signersThreshold;
    if (snapshotThreshold == 0 || _liveThreshold < snapshotThreshold) {
        return false;
    }
    uint256 stillOwners = 0;
    uint256 signerCount = teeGov.signers.length();
    for (uint256 i = 0; i < signerCount; i++) {
        if (_safe.isOwner(teeGov.signers.at(i))) {
            stillOwners++;
            if (stillOwners >= snapshotThreshold) {
```

```

        return true;
    }
}
return false;
}

```

However, this establishes only that the snapshot could still be satisfied, and both values it consults, `getThreshold()` and `isOwner(...)`, are read live from the Safe and are under the Safe's own control. Nothing ties the identities that actually confirmed the Safe transaction to the frozen snapshot's membership, so a Safe whose ownership has diverged from the snapshot can satisfy it by reconfiguring itself beforehand.

Consider a Safe snapshotted with owner set $\{A, B, C\}$ and threshold two, producing a governance hash that a set of machines binds. Ownership is later changed to $\{D\}$ with threshold one, leaving D in sole control of the Safe. D remains able to pass the check for the original snapshot through a sequence of ordinary authorized self-calls:

1. Rotate the owner set to addresses D holds, $\{D, D1, D2\}$, and raise the threshold to three.
2. Add A and B back to the owner set, giving $\{D, D1, D2, A, B\}$.
3. Execute `approveMachinePathList`. The live threshold of three is not below the snapshot threshold of two, and two snapshot signers, A and B, are live owners, so `isSnapshotSatisfiable` returns true and the snapshot is marked `safeApproved`. The transaction itself is confirmed by D, D1, and D2, none of which belong to the snapshot.
4. Restore the owner set to $\{D\}$ with threshold one.

Impact

Any party controlling the Safe's live configuration can satisfy a historical snapshot it was never a member of, so an active machine path list does not carry threshold-many approvals from each involved snapshot.

Recommendations

Do not derive authorization from the Safe's live configuration. Reconstruct the EIP-712 Safe transaction hash for the approval call and require that at least snapshot-threshold-many signatures over it recover to members of the frozen snapshot.

Remediation

This issue has been acknowledged by Flare Foundation, and a fix was implemented in commit [aaa2eeb1](#).

3.4. Stale healthy availability proofs can suspend production TEE machines when challengeValidityDurationSeconds is configured too high

Target	MachineManagerFacet		
Category	Business Logic	Severity	Low
Likelihood	Low	Impact	Low

Description

The permissionless pauseWithProof function in MachineManagerFacet suspends a production TEE machine when its reported status is not OK or Verification.verifyAvailabilityCheckProof() returns false.

```
function pauseWithProof(
    ITeeAvailabilityCheck.Proof calldata _proof
)
external
{
    // [...]
    bool responseDataValid = Verification.verifyAvailabilityCheckProof(
        teeMachine, state.status, _proof
    );
    require(
        !responseDataValid ||
        _proof.responseBody.status !=
        ITeeAvailabilityCheck.AvailabilityCheckStatus.OK,
        InvalidResponseDataOrAvailabilityCheckStatus()
    );
    // [...]
}
```

However, verifyAvailabilityCheckProof() evaluates responseBody.lastSigningPolicyId against the reward epoch at submission time, rather than the reward epoch applicable when the proof was produced.

```
uint256 lastSigningPolicyId = _responseBody.lastSigningPolicyId;
if (lastSigningPolicyId != _currentRewardEpochId && lastSigningPolicyId !=
    _currentRewardEpochId + 1) {
    return false;
}
```

For example, a proof produced during reward epoch N with `lastSigningPolicyId == N` and `status == OK` passes verification during epoch N. If reward epoch N+1 begins while the associated challenge remains valid, submitting the same proof causes the signing-policy check to return `false`. Although the proof demonstrated that the machine was healthy when it was produced, `pauseWithProof()` interprets its submission-time staleness as evidence of failure and changes the machine's status from `PRODUCTION` to `SUSPENDED`.

This condition can occur when `challengeValidityDurationSeconds` exceeds the guaranteed lead time for delivering the next signing policy. The deployment configurations set the challenge window to 600 seconds, but governance can configure it as high as one day.

```
function updateSettings(  
    uint64 _availabilityCheckValidityDurationSeconds,  
    uint64 _signingPolicyValidityDurationInRewardEpochs,  
    uint64 _challengeValidityDurationSeconds  
)  
    internal  
{  
    // [...]  
    validateDuration(_challengeValidityDurationSeconds, 1 minutes, 1 days);  
    // [...]  
}
```

Impact

If the challenge duration is configured above the guaranteed next-policy delivery lead time, an attacker can submit a stale but challenge-valid proof to suspend a healthy production TEE machine. Suspension removes the machine from the active sets, potentially disrupting operations and causing its operator to miss rewards until a fresh proof returns the machine to production.

Recommendations

Lower the upper bound for `challengeValidityDurationSeconds` from one day to a value safely below the lead time for delivering the next signing policy.

Remediation

This issue has been acknowledged by Flare Foundation, and a fix was implemented in commit [c2a90c47](#).

3.5. Freshly registered TEEs cannot restore older backups

Target	WalletBackupManagerFacet		
Category	Business Logic	Severity	Low
Likelihood	Medium	Impact	Low

Description

The admin-threshold restore flow is documented as the mechanism for recovering a wallet key from an existing backup onto a freshly registered TEE. `KeyManagement.md` states that when a TEE custodying a wallet has failed, "the keys can be restored to a freshly-registered TEE via the admin-threshold path," and that this path is entered through `WalletBackupManagerFacet.backupRestore`.

When a TEE first transitions from `INITIALIZED` to `PRODUCTION`, `MachineManagerFacet.toProduction` records the signing-policy ID reported by its initial availability proof as the machine's `initialSigningPolicyId`.

```
if (status == TeeStatus.INITIALIZED) {  
    state.initialSigningPolicyId = _proof.responseBody.initialSigningPolicyId;  
}
```

This anchor is immutable thereafter and, because signing policies track reward epochs, it is effectively the reward epoch in which the machine first became trusted. A freshly registered TEE therefore carries an `initialSigningPolicyId` close to the current reward epoch.

However, `WalletBackupManagerFacet._validateRestoreInputs` requires the destination TEE's `initialSigningPolicyId` to be less than or equal to the backup's `rewardEpochId`.

```
require(  
    MachineManager.getInitialSigningPolicyId(_destinationTeeId)  
    <= _backupId.rewardEpochId,  
    UnsupportedRewardEpochId()  
);
```

Consequently, a TEE registered in reward epoch N cannot restore any backup created before epoch N , even when the current signing policy can still decrypt a sufficient threshold of the old backup shares. This is the typical ordering for disaster recovery, where the backup predates the replacement TEE: the machines whose `initialSigningPolicyId` is old enough to satisfy the check are exactly the ones that were already in production when the backup was taken, which in a recovery scenario are the machines that have failed. The check therefore prevents the

documented recovery flow from restoring older, otherwise usable backups onto a fresh machine.

The same restriction applies to `directRestore`, because `backupRestore` and `directRestore` share `_validateRestoreInputs`.

Impact

A restore to a freshly registered TEE reverts whenever the backup predates that TEE's registration, so the documented recovery flow does not work as described for the common disaster-recovery ordering.

Recommendations

Relax the `initialSigningPolicyId <= rewardEpochId` check so that an old backup can be restored to a freshly registered TEE, as long as the current signing policy can still decrypt the required threshold of its shares.

Remediation

This issue has been acknowledged by Flare Foundation, and a fix was implemented in commit [87ba6b40](#).

3.6. Failed availability proofs are not invalidated by a later successful availability confirmation

Target	MachineManagerFacet		
Category	Business Logic	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

A proof with a non OK status can be submitted to the permissionless `pauseWithProof` function to suspend a production TEE machine. If such a proof is not used right away, a later successful `confirmAvailability` does not invalidate it. This is because `confirmAvailability` does not update the machine's `lastStatusChangeTs`, which is the only freshness check that `pauseWithProof` applies to the proof.

```
MachineManager.validateAvailabilityCheckTs(teeId, _proof.header.timestamp);
```

As a result, an old failed proof can still be used to suspend a machine that has since proven it is healthy.

Impact

This flow is not reachable in practice. The `DOWN` value in the `AvailabilityCheckStatus` enum is a leftover that was forgotten in the interface. The off-chain verifier only returns `OK` or `OBSOLETE`.

Recommendations

Remove the unused `DOWN` value from the `AvailabilityCheckStatus` enum so the interface only lists the statuses the off-chain verifier can return, and document that `OBSOLETE` is final.

Remediation

This issue has been acknowledged by Flare Foundation, and a fix was implemented in commit [cfe06195](#).

4. System Design

This provides a description of the high-level components of the system and how they interact, including details like a function's externally controllable inputs and how an attacker could leverage each input to cause harm or which invariants or constraints of the system are critical and must always be upheld.

Not all components in the audit scope may have been modeled. The absence of a component in this section does not necessarily suggest that it is safe.

4.1. FlareTeeManager: TEE machine lifecycle

Description

This component manages the lifecycle of TEE machines inside the FlareTeeManager diamond. A machine registers, proves it is alive through FDC2 attestation, and then enters production. Machines that fail availability checks are suspended, and owners or extension owners can pause or ban them. Only production machines can hold wallet keys and receive instructions.

MachineManagerFacet.register

The `register` function onboards a new TEE machine into the FlareTeeManager diamond. The caller must be the declared owner and be on the extension's machine owner allowlist. The machine proves possession of its enclave key by signing its registration data. The machine is stored in the `INITIALIZED` state, and an FDC2 attestation challenge is sent to it. It gains no privileges until the attestation proof is verified later in `toProduction`.

MachineManagerFacet.toProduction

The `toProduction` function moves a machine into the `PRODUCTION` state using an FDC2 availability check proof. The owner must call it for `INITIALIZED` or `PAUSED` machines, while anyone can recover a `SUSPENDED` one. It checks that the machine's code hash and platform are still enabled and that the proof answers the current challenge. On first activation it also records the machine's initial signing policy ID. On success the machine becomes `PRODUCTION` and its availability window is extended.

MachineManagerFacet.pause

The `pause` function takes a machine out of production without requiring a proof. The owner can always pause a `PRODUCTION` or `SUSPENDED` machine, and so can anyone if its version was disabled. Any other caller can only suspend a production machine whose availability check has expired. That third party path is blocked while the extension is in emergency pause or its grace period.

MachineManagerFacet.pauseWithProof

The `pauseWithProof` function suspends a production machine using an FDC2 availability check proof. Anyone can call it when the proof reports a failed check or its response data does not verify. The proof must be at least as recent as the machine's current challenge. On success the machine moves to the `SUSPENDED` state, from which anyone can restore it via `toProduction`.

MachineManagerFacet.ban and MachineManagerFacet.unban

The `ban` function lets the extension owner remove a machine from service in any active state. A banned machine cannot change state through any other lifecycle function. The `unban` function, also restricted to the extension owner, returns the machine to the `PAUSED` state. From there the machine must pass a fresh availability check to reach production again.

VerificationFacet.requestTeeAttestation

The `requestTeeAttestation` function sends an attestation instruction to a registered machine. It reuses the machine's stored challenge if still valid, and otherwise derives a new one from the Relay's random number. Anyone can call it, since it only asks the machine to prove itself. The claim back address receives any unused portion of the attestation fee.

VerificationFacet.requestAvailabilityCheckAttestation

The `requestAvailabilityCheckAttestation` function asks FDC2 to attest that a machine answered its current challenge. It reverts if the machine's challenge has already expired. For machines still in the `INITIALIZED` state, the governance set registration cosigners are included in the request. The resulting proof is consumed by `toProduction`, `confirmAvailability`, or `pauseWithProof`.

VerificationFacet.confirmAvailability

The `confirmAvailability` function extends a production machine's availability window using a passing FDC2 proof. It requires the machine to be in production with its code hash and platform still enabled. The proof is verified against the machine's attestation data before the window is extended. If the window is allowed to expire, anyone can suspend the machine through `pause`.

4.2. FlareTeeManager: wallet key management

Description

This component manages the signing keys that TEE machines generate and hold for wallets. The wallet project owner adds keys on production machines and confirms them with TEE-signed existence proofs. Keys can be copied between machines through data provider backups or direct

transfers along governance-signed paths. Per-TEE key nonces protect the delete and restore flows against replayed proofs.

WalletKeyManagerFacet.addKey

The `addKey` function asks a production TEE machine to generate a new key for a wallet. Only the project owner can call it, and the wallet must be in the `INITIALIZED` state. The machine must belong to the same extension as the wallet's project. It assigns the next key ID and sends a `KEY_GENERATE` instruction carrying the wallet's admin and cosigner configuration. The key is not usable until the machine's existence proof is accepted by `confirmKey`.

WalletKeyManagerFacet.confirmKey

The `confirmKey` function registers a key on chain using an existence proof signed by the holding TEE. The proof must match the key's per TEE nonce, the project's key type and signing algorithm, and the wallet's admin and cosigner configuration. The TEE's signature over the proof must recover to the machine's identity address. A first confirmation must come from the project owner with a freshly generated key, and stores the public key. Later confirmations must present a restored key with the same public key, and add the new machine as a holder.

WalletKeyManagerFacet.deleteKey

The `deleteKey` function removes a machine from a key's holder list and tells it to erase the key. Only the project owner can call it, and the machine must be in production within the wallet's extension. It sends a `KEY_DELETE` instruction carrying the key's incremented per TEE nonce. The nonce bump invalidates any existence proof the machine produced earlier, so the key cannot be reconfirmed there.

WalletBackupManagerFacet.backupRestore

The `backupRestore` function restores a key onto a production machine from a data provider backup URL. Only the project owner or the project's backup manager can call it. The destination must not already hold the key, and the backup's public key and reward epoch are validated. It bumps the destination's per TEE key nonce and sends a `KEY_DATA_PROVIDER_RESTORE` instruction. The instruction carries the wallet admins and their threshold, so the restore needs admin approval inside the TEE.

WalletBackupManagerFacet.directBackup

The `directBackup` function tells a source machine to generate a backup blob encrypted for a destination machine. Only the project owner or backup manager can call it, and both machines must be production members of the project's extension. The transfer must follow a source to destination pair present in the extension's active governance signed path list. The source must

hold the confirmed key and the destination must not hold it yet. It sends a `KEY_DIRECT_BACKUP` instruction with no extra cosigners, since the signed path list is the authorization.

WalletBackupManagerFacet.directRestore

The `directRestore` function imports a direct backup blob on the destination machine. Only the project owner or backup manager can call it, with the same validation as other restores. The source to destination pair must still be present in the extension's active path list. It bumps the destination's per TEE key nonce, so a stale restore attestation cannot be replayed later.

4.3. MachinePathManagerFacet path list flow

Description

The machine path list flow defines which direct key transfers TEE governance has approved for an extension. The extension owner or operator creates a numbered list with `createNewMachinePathList` and fills it using `addMachinePaths`. `finalizeMachinePathList` freezes the list into a message hash that binds the extension ID, list nonce, and chain ID against replay. Governance signers then approve it through `signMachinePathList`, each signature counted against every governance set they belong to. Once all involved governance sets reach their thresholds, the list becomes the extension's active path list.

4.4. FlareTeeManager: instruction dispatch

Description

Instructions are how the contracts communicate work to TEE machines, which observe the emitted events off-chain. The `InstructionsFacet` exposes two entry points, one for extension senders and one for governance registered system senders. Both paths share logic that validates machines, collects fees, and emits the instruction event.

InstructionsFacet.sendInstructions

The `sendInstructions` function routes an instruction from an extension's registered sender to its TEE machines. The target extension is derived from the first TEE ID, and duplicates are removed from the list. Callers that are not system senders must match the extension's registered instructions sender. They are also forbidden from using the reserved `F_system` operation type prefix. The attached fee is forwarded and a fresh instruction ID is generated and returned.

InstructionsFacet.sendSystemInstructions

The `sendSystemInstructions` function is restricted to system sender contracts registered by governance. It lets system components like the attestation flow dispatch reserved `F_system` operations.

Unlike the regular path, the caller may supply its own instruction ID instead of generating one.

Instructions.sendInstructions and Instructions.generateInstructionId

The library function `Instructions.sendInstructions` is the single function shared by both entry points. It requires all target machines to belong to one extension and blocks dispatch while that extension is emergency paused. Machines must be in production, except for system operation types, which are also delivered during registration. The fee is the per operation fee times the machine count, and the entire payment goes to the reward manager. The `generateInstructionId` function derives unique IDs from the extension ID, a per extension counter, and the previous block hash.

4.5. TeePayments: payment contracts

Description

The TeePayments contracts let wallet owners instruct payments from TEE-controlled multisig accounts on external chains. The account model targets chains with sequential nonces, while the UTXO model groups payments into anchor-bound batches. Both models share `TeePaymentsBase` and dispatch PAY and REISSUE system instructions through the `FlareTeeManager` diamond. A governance registry binds each source chain to exactly one payment contract.

TeePayments.pay

The `pay` function submits a single payment instruction for an account model multisig account. Only the account's authorization address can call it, the wallet must be in production, and the recipient address must validate. It assigns the next payment ID, stores the payment hash, and derives the native chain nonce from the initial nonce. The message includes the wallet's receiving TEEs, cosigners, and the effective fee schedule. It is sent to the TEEs as a PAY system instruction through the diamond.

TeePayments.reissue

The `reissue` function resends a previously submitted payment that might have failed (e.g. not enough fee). The caller must be the authorization address, and the supplied instruction must match the stored payment hash. A per-payment counter gives every reissue a distinct instruction ID, starting at one. The account model reissues one payment per call, so the reissue always finalizes.

TeePayments.addPMWMultisigAccount

The `addPMWMultisigAccount` function adds an external chain account to a wallet using an FDC2 configuration proof. Only the wallet's project owner can call it, and the project must belong to the system extension with a matching key type. The proof is validated by the config verifier before any

of its fields are trusted. The account's initial nonce is taken from the proof's sequence, and payment IDs start at one. An account hash can only ever be bound to one wallet.

TeePaymentsUtxo.pay

The `pay` function adds a payment to the account's open batch, opening a new batch when needed. Opening a batch selects the next free anchor in round robin order and reserves it until the batch ends. Only the authorization address can call it, the wallet must be in production, and the recipient address must validate. The batch closes when it fills, its duration passes, or the reward epoch changes. Each payment is sent as a PAY instruction whose ID is shared by the whole batch.

TeePaymentsUtxo.reissue

The `reissue` function replaces the payments of a closed batch, possibly spread over several calls. Supplying the batch's first payment again starts a fresh replacement attempt with a new attempt ID. Every supplied instruction must match its stored payment hash, and the attempt must stay within one reward epoch. The attempt finalizes once every payment in the batch has been reemitted.

TeePaymentsUtxo.addPMWMultisigAccount

The `addPMWMultisigAccount` function registers a UTXO account and its initial anchors from an FDC2 configuration proof. Only the wallet's project owner can call it, under the same system extension and key type checks as the account model. The proof supplies the account address, account index, and up to 32 genesis anchor UTXOs. Batching starts conservatively with a batch size of one and no duration.

TeePaymentsUtxo.addAnchors

The `addAnchors` function extends a registered account's anchor list using a fresh FDC2 configuration proof. Only the owner of the account's wallet project can call it. The proof must repeat all existing anchors exactly and add at least one new one. New anchors are immediately available, giving the account more parallel batch capacity.

4.6. TeePaymentsUtxo batch settings

Description

The `setBatchSettings` function lets the wallet owner choose an account's batch size and duration, applied from the next batch. Effective values are capped governance, set through `setMaxBatchSettings`. Batches cannot open at all until governance has configured a nonzero maximum batch size for the source. The `setAnchorReuseDelay` function lets governance set how long an anchor stays reserved after its batch ends.

TeePaymentsRegistry.registerSources

The registerSources function lets governance bind source chains to a payment contract with its op type, key type, and model. The target contract must be deployed code and must report the same payment model as the registration claims. A source stays bound to one contract until governance unregisters it. All payment contracts consult this registry to reject calls for sources they do not own.

4.7. TeePaymentsFeeScheduleManager schedules

Description

The fee schedule manager stores fee bump schedules as factor and delay entries, applied by TEEs after a payment stalls. Governance first sets per-source limits on entry count and delay, and only account model sources can be configured. The project owner sets a project default with setProjectFeeSchedule and per-account overrides with setAccountFeeSchedule. The getEffectiveSchedule function resolves the account override first, then the project default, then a built-in default. Ownership checks derive the project from the account itself, so owners cannot touch other projects' schedules.

4.8. TeePaymentsConfigVerifier proof verification

Description

The verifyAccountConfiguredProof and verifyUtxoConfiguredProof functions validate FDC2 proofs that an external account matches a wallet. They check the attestation type and that the proof's public keys and threshold equal the wallet's current keys. The proof must carry valid TEE or signing policy signatures plus the governance-set cosigner signatures. The response status must be OK, and UTXO proofs are limited to 32 anchors. The verifier holds no account state; the payment contracts read the verified fields directly from the proof.

5. Assessment Results

During our assessment on the scoped FSP diff - FCC contracts, we discovered six findings. No critical issues were found. Three findings were of medium impact, two were of low impact, and the remaining finding was informational in nature.

5.1. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Where Zellic states that a finding has been addressed or fixed in one or more specific commits, this indicates only that those commits introduce changes that, in our assessment, address the finding relative to the codebase version originally reviewed. This does not imply that Zellic reviewed other changes contained in those commits, the overall state of the codebase at those commits, or the interplay between remediation measures for different findings.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.